

Elastic *Stack*

Bilaketa, behagarritasuna eta log-en kudeaketa

Zikloa Adimen Artifiziala eta Big Data

Modulua Big Data Sistemak



Elastic Stack

-
- 01 Sarrera: log kaosa eta pipeline-a
 - 02 Elasticsearch: alderantzizko indizea eta datu-eredua
 - 03 Query DSL: kontsultak eta agregazioak
 - 04 Kibana: bistaratzea eta ILM
 - 05 Ingesta: Beats eta Logstash (grok)
 - 06 Ariketak eta ondorioak

01 Sarrera eta arkitektura

“

Gaueko 3:00etan bezero batek deitu du: webgunea erori da. 50 zerbitzari daude. SSH, grep, SSH, grep... ordu bat igaro da. Oraindik ez dakigu non da arazoa.

— ELASTIC STACK-EN ZERGATIA · LOG KAOSA

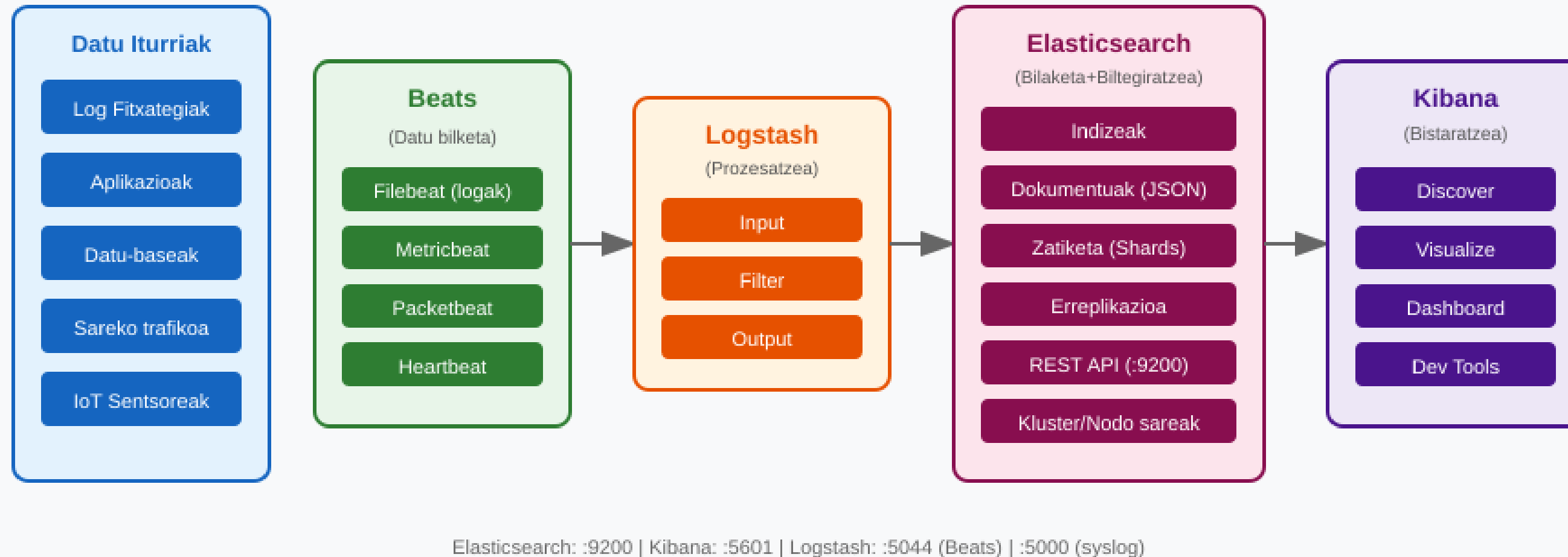
Zer da Elastic Stack?

- Kode irekiko plataforma datu kopuru handiak **biltzeko, bilatzeko, aztertzeko eta bistaratzeko**
- Historikoki **ELK Stack: Elasticsearch + Logstash + Kibana**
- Beats gehitu ondoren, **Elastic Stack** izena hartu zuen
- Erabilera nagusiak: **bilaketa, behagarritasuna, segurtasuna, IoT eta IA**
- Ez da soilik log-ak kudeatzeko sistema bat

— ARKITEKTURA

Elastic Stack: Datu-Fluxua

Iturria → Beats → [Logstash] → Elasticsearch → Kibana. Logstash aukerakoa da (datuak egituratuta badaude, zuzenean ES-era).



Hiru bloke nagusi

- **Observabilitatea** — sistemetan zer gertatzen ari den ikustea
logak, metrikak eta aplikazioen errendimendua
- **Segurtasuna** — Segurtasun-gertaerak aztertzea edo portaera susmagarriak eta iruzurra detektatzea
- **Bilaketa eta datuen erabilera** — adibidez, e-commerce batean milioika produkturen artean erabiltzaileak nahi duena oso azkar aurkitzea

Elastic Stack: Erabilera-kasuak

A. Observabilitatea

4 erabilera-kasu

-  Log Analisi Zentralizatua
-  APM — Errendimendu Monitorizazioa
-  Sistema Monitorizazioa (CPU/RAM)
-  LLM / AI Observability

B. Segurtasuna

3 erabilera-kasu

-  SIEM — Segurtasun Analisia
-  Iruzur Detekzioa
-  Zibersegurtasun Analitika

C. Datu Gestio & AI

5 erabilera-kasu

-  E-commerce Bilaketa
-  Mantenu Prediktiboa (IoT)
-  Analisi Geospazialak
-  Osasungintza
-  RAG eta Vektore-bilaketa

4 Osagaiak · Rol Bakoitza

- **Elasticsearch** — Datuak gorde, indexatu eta bilatzeko motor banatua. Stack-aren **bihotza**. REST API: **:9200**
- **Logstash** — Datuak jaso, eraldatu eta bidaltzeko pipelinea: **INPUT → FILTER → OUTPUT**
- **Kibana** — Elasticsearch-eko datuak kontsultatu, aztertu eta bistaratzeko web interfazea. **:5601**
- **Beats** — Datu-iturburuetan instalatzen diren **agente arinak**, datuak jaso eta bidaltzeko.

— KONPARAKETA

Noiz aukeratu Elastic?

Elastic Stack vs Alternatiboa

	Elastic Stack	Splunk	Grafana+Loki	SQL DB
Log analisia	✓	✓	✓	⚠
Testu bilaketa	✓	✓	✓	✗
SIEM segurtasuna	✓	✓	⚠	✗
Mantenu predikt.	✓	⚠	⚠	✗
Geospazialak	✓	✓	⚠	✓
RAG / AI	✓	⚠	⚠ ⚠	✗
Kostua	● Kode irekia	● Oso garestia	● Doakoa	● Merkea
Erraztasuna	● Ertaina	● Erraza	● Erraza	● Erraza

- **Splunk** — Antzeko erabilera-esparru zabala: logak, behagarritasuna eta segurtasuna. **Komertziala**, hainbat lizentzia/preczo eredurekiin.
- **Grafana + Loki** — Logen bilketa, bilaketa eta bistaratzeko aukera indartsua; Loki-k batez ere metadata/labels indexatzen ditu eta kostu txikiagoa bilatzen du.
- **SQL DB** — Datu egituratuak eta **transakzioak (OLTP)** kudeatzeko aproposa; ez da Elastic-en ordezeko zuzena testu-bilaketa masiborako.
- **Elastic Stack** — Bilaketa + log analisia + behagarritasuna + segurtasuna + bilaketa bektoriala/AI plataforma berean.

— 1. PRAKTIKA

ELASTICSEARCH + KIBANA MARTXAN JARRI

HELBURUA:

Docker Compose erabiliz **Elasticsearch** eta **Kibana** abiarazi eta bi zerbitzuak erabilgarri daudela egiaztatu.

PAUSOAK:

1. Sortu lanerako karpeta, **compose.yaml** fitxategiarekin
2. Ingurunea abiarazi
3. Probatu nabigatzailean:
 - Elasticsearch -> localhost:9200
 - Kibana → localhost:5601
4. Zer erakusten du Elasticsearch-ek localhost:9200 irekitzean?
Zer desberdintasun dago Kibana irekitzean?

```
compose.yaml
1  services:
2
3      elasticsearch:
4          image: docker.elastic.co/elasticsearch/elasticsearch:9.5.4
5          container_name: iabd-elasticsearch
6          environment:
7              - discovery.type=single-node
8              - xpack.security.enabled=false
9          ports:
10             - "9200:9200"
11          volumes:
12             - elasticsearch_data:/usr/share/elasticsearch/data
13
14      kibana:
15          image: docker.elastic.co/kibana/kibana:9.5.4
16          container_name: iabd-kibana
17          ports:
18             - "5601:5601"
19          environment:
20             - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
21          depends_on:
22             - elasticsearch
23
24  volumes:
25      elasticsearch_data:
```

02 Elasticsearch

Zergatik da Elasticsearch azkarra?

SQL-ren arazoa:

- Datu kopuru handietan **garestia** izan daiteke.
- Ez dago testu-bilaketarako bereziki optiizatuta.
- Ez du berez **errelevantzia-puntuaziorik**.
- **LIKE**-ek ez ditu berez antzeko hitzak edo akatsa ortografikoak kudeatzen.

Elasticsearch:

- Dokumentuak sartzean **alderantzizko indizea** eraikitzen du.
- Bilaketa **indize horren gainean** egiten du.
- Datuak **JSON dokumentu** gisa kudeatzen ditu.
- **REST API** bidez kontsultatzen da.
- **_score** → emaitzak errelevantziaren arabera ordenatzeko

GAKOA: ALDERANTZIZKO INDIZEA

Alderantzizko indizea

Liburu baten aurkibidea bezala: **hitza**
→ **zein dokumentutan** dagoen.

Bilaketa indizean egiten da, ez
datuetan — millisegundotan.

Elasticsearch: Alderantzizko Indizea

Dokumentuak

Doc 1: "Elasticsearch bilaketa azkarra"
id=1

Doc 2: "Kibana bistaratzea azkarra"
id=2

Doc 3: "Logstash datuak bilaketa"
id=3

Indexatu

Alderantzizko Indizea

Hitza (Term)	Dokumentuak
bilaketa	Doc 1, Doc 3
azkarra	Doc 1, Doc 2
elasticsearch	Doc 1
kibana	Doc 2
bistaratzea	Doc 2
logstash	Doc 3
datuak	Doc 3

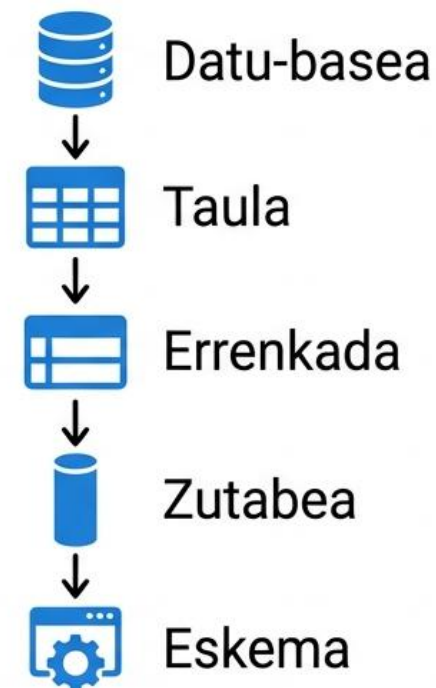
Bilaketa "bilaketa" → Doc 1 eta Doc 3 berehala aurkitu

Indizea, dokumentua, mapaketa

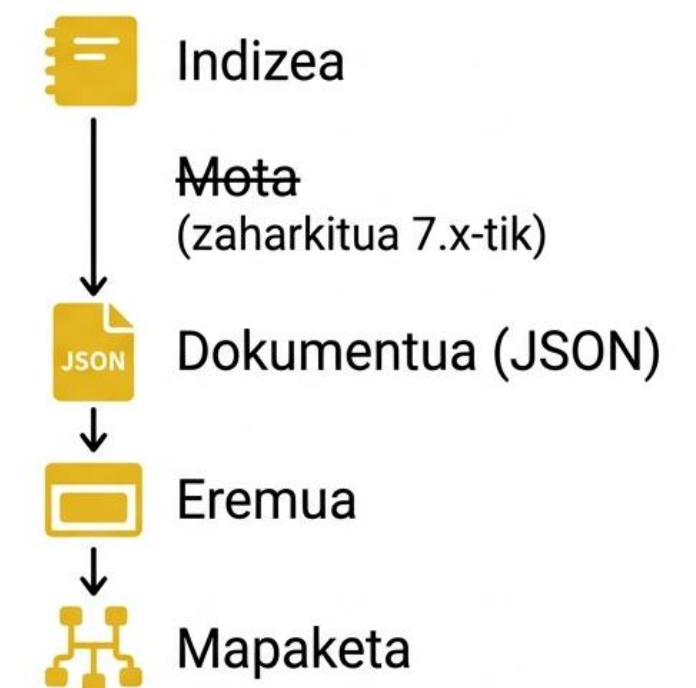
- Indizea ≈ taula
- Dokumentua ≈ errenkada → JSON
- Eremua ≈ zutabe
- Mapaketa (mapping) ≈ eskema
- ⚠ Ez da SQLko JOIN ereduan oinarritzen — desnormalizatu (dena dokumentu berean)
- Datu erlazionatuak dokumentu berean gordetzea → bilaketa sinpleago eta azkarragoa

Datu-eredua: Erlazionala vs Elasticsearch

Datu-base Erlazionala



Elasticsearch



⚠ ES-ek ez du JOIN operadorerik — harreman konplexuak aplikazio-mailan konpondu

Elasticsearch: Eremu Motak

⚡ **text** vs **keyword**: akats ohikoena!



text

Testu osoko bilaketa · Tokenizatua ·
match kontsultak · Ezin ordenatu
Koaderno gorria arina



keyword

Balio zehatza · Ez tokenizatua ·
term kontsultak · Ordenatu/Agregatu ✓
bulegoa



integer / long

Zenbaki osoak · range kontsultak
42



float / double

Zenbaki hamartarrak
2.99



date

Data eta orduak · ISO8601 formatua ·
now-1h espresioak
2024-01-15



boolean

Egia/Faltsua · Iragazketa sinplea
true / false



geo_point

Lat/Lon kokapena · Gertuko bilaketa ·
Geo-fencing
{lat: 43.26, lon: -2.92}



nested

Objektu-zerrenda · Harreman konplexuak
[{...}, {...}]

— EREMU-MOTAK

text vs keyword

text — tokenizatua, `match`
bilaketarako (testu librea).

keyword — bere horretan, `term`
bilaketarako (iragazi, ordenatu,
agregatu).

— 2. PRAKTIKA

LEHEN INDIZEA ETA DOKUMENTUAK

HELBURUA:

Kibana **Dev Tools** erabiliz Elasticsearch-en indize bat sortu, JSON dokumentuak txertatu eta datuak kontsultatu.

PAUSOAK:

1. Ireki Kibana → Dev Tools

2. Sortu lehen dokumentua:

```
1 POST produktuak/_doc
2 {
3   "izena": "Koaderno urdina",
4   "kategoria": "Papergintza",
5   "prezioa": 4.50,
6   "stock": 25
7 }
```

3. Gehitu beste 3 dokumentu:

```
POST produktuak/_doc
{
  "izena": "Koaderno handia",
  "kategoria": "Papergintza",
  "prezioa": 8.50,
  "stock": 10
}
```

```
POST produktuak/_doc
{
  "izena": "Sagu optikoa",
  "kategoria": "Informatika",
  "prezioa": 19.99,
  "stock": 15
}
```

```
POST produktuak/_doc
{
  "izena": "Teklatu mekanikoa",
  "kategoria": "Informatika",
  "prezioa": 59.99,
  "stock": 5
}
```

4. Kontsultatu indizeko dokumentuak:

```
GET produktuak/_search
```

5. Aztertu emaitza:

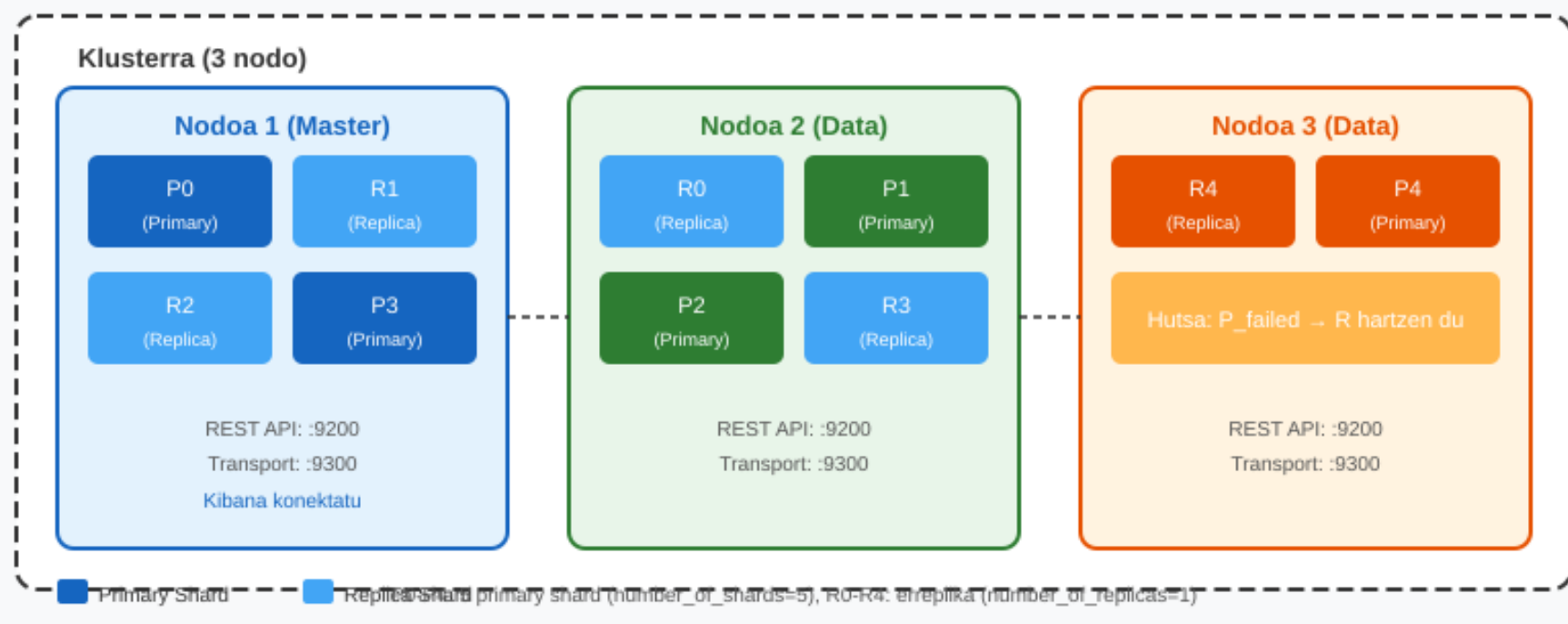
- Zenbat dokumentu daude?
- Non agertzen dira dokumentuaren benetako datuak?
- Zer da produktuak?
- Zer da { "izena": ..., "prezioa": ... } bakoitza?

— ESKALA

Shards eta replicas

- **Shard** — indizearen zati bat. Datuak nodoen artean **banatzeko** erabiltzen da → eskala horizontala
- **Replica** — shard baten **kopia**. Erabilgarritasuna eta bilaketa-errendimendua hobetzen ditu.
- ⚠ **Primary shard kopurua** indizea sortzean definitzen da; aldatzeko normalean **reindexatu** behar da. **Replica kopurua alda daiteke**.
- **Klusterraren osasuna:**
 - ● GREEN – primary eta replica guztiak esleituta
 - ● YELLOW – primary guztiak esleituta, baina replica batzuk ez
 - ● RED – primary shard bat edo gehiago ez dago erabilgarri

Elasticsearch: Zatiketa (Shards) eta Erreplikazioa



— 3. PRAKTIKA

CLUSTER HEALTH ETA MAPPING-A AZTERTU

HELBURUA: Elasticsearch klusterraren egoera aztertu eta **produktuak** indizearen **shard-ak** eta **mapping-a** ikusi.

PAUSOAK:

1. Klusterraren egoera aztertu.

```
GET _cluster/health
```

- Identifikatu: **status**, **number_of_nodes**, **active_primary_shards**, **active_shards**

2. **produktuak** indizearen shard-ak aztertu:

```
GET _cat/shards/produktuak?v
```

- Identifikatu: Primary shard-a (**p**), Replica (**r**), **STARTED** / **UNASSIGNED** egoerak

4. Indizearen mapping-a kontsultatu.

```
GET produktuak/_mapping
```

- Identifikatu Elasticsearch-ek automatikoki esleitutako datu-motak: izena, kategoria, prezioa, stock

5. Erantzun:

- a. Zein da klusterraren egoera? Zergatik?
- b. Zer gertatzen da replica-rekin? Zergatik?
- c. Zein datu-mota esleitu dio Elasticsearch-ek eremu bakoitzari?
- d. **izena** eta **kategoria** eremuetan, zergatik agertzen dira **text** eta **keyword**?

03

Query DSL eta agregazioak

Query DSL: Elasticsearch-en kontsultatzeko lengoaia

JSON formatuan idazten da → SQL-ko **WHERE**-en antzeko funtzioa.

Bi kontsulta-familia nagusi:

- **FULL-TEXT** → **text** eremuetarako
 - **match** → testu-bilaketa
 - **multi_match** → hainbat eremutan bilatu
 - **match_phrase** → esaldi/hurrenkera zehatza bilatu
 - Errelebantzia kalkulatu du → **_score**
- **TERM-LEVEL** → balio zehatz, zenbaki eta datetarako
 - **term** / **terms** → balio zehatzak
 - **range** → balio-tarteak (> , < , datak...)

Full-text vs Term

Elasticsearch: Query DSL Kontsulta Motak

Full-text Kontsultak

text eremuetan · tokenizatua · `_score` kalkulatu



match

Eremu batean hitz-bilaketa (OR/AND)



multi_match

Hainbat eremutan aldi berean (pisuekin)



match_phrase

Esaldi osoa, hurrenkera berean

Erabilera: testu libreko bilaketa

Term Kontsultak

keyword/zenbaki/data · zehatza · cacheatua



term

Balio zehatza



terms

Balio-zerrenda (IN)



range

Tartea: gte, lte, gt, lt



exists

Eremu existentzia



wildcard/fuzzy

Patroiak eta akats-tolerantzia

Erabilera: iragazketa zehatza

Konposatutako Kontsultak

beste kontsultak konbinatu



bool

must + should + must_not + filter

- **must**: beharrezkoa + `_score`
- **should**: aukerakoa + `_score` hobetu
- **must_not**: baztertu
- **filter**: beharrezkoa + cacheatua ⚡

Erabilera: kontsulta konplexuak

Full-text (`match``) — text eremuetan, analizatua, `_score``.

Term (`term``, `range``) — balio zehatza, ez tokenizatu.

💡 **filter > must** errendimenduarentzat — ez du `_score` kalkulatu eta cacheatzen da

Bool Kontsulta: Lau Klausulak

must ✓

Dokumentua **BETE BEHAR** du
_score-an **ERAGINA DU** ✓

```
{ "match": { "izena": "koaderno" } }
```

SQL: **"AND"**

should 🤔

🤔 Dokumentua bete **DEZAKE** **HOBETU** egiten du **?**
_score **HOBETU**
minimum_should_match: 1

```
{ "term": { "kategoria": "bulegoa" } }
```

SQL: **"OR (puntuazioa hobetu)"**

bool

must_not ✗

Dokumentua **EZ** du bete behar
Ez du _score-an eraginik ✗

```
{ "term": { "eskuragarri": false } }
```

SQL: **AND NOT**

filter ⚡

⚡ Dokumentua **BETE BEHAR** du
Ez du _score-an eraginik
CACHEATUA → AZKARRAGOA ⚡

```
{ "range": { "prezioa": { "gte": 1 } } }
```

SQL: **AND (optimizatua)**

⚡ **Arau praktikoa:** Zenbakizko iragazketak eta datak BETI filter barruan — must baino azkarragoa

— TRESNA NAGUSIA

Bool kontsulta – 4 klausula

Bool kontsulta hainbat kontsulta edo baldintza **konbinatzeko** erabiltzen da.

- **must** → bete behar da
- **should** → lehentasuna eman
- **must_not** → baztertu
- **filter** → iragazi, _score-an eragin gabe

must vs filter — errendimendua

	MUST	FILTER
Bete behar du	Bai	Bai
_score kalkulatu	Bai	EZ
Cacheatua	Ez	Bai ⚡
Noiz	Testu-bilaketa	Zenbaki, data, balio zehatza

— 4. PRAKTIKA

QUERY DSL: LEHEN BILAKETAK

HELBURUA: Query DSL erabiliz **produktuak** indizean **testu-bilaketak**, **prezio-iragazkiak** eta **baldintza konbinatuak** egitea

PAUSOAK:

1. Bilatu izenean **koaderno** hitza duten produktuak:

```
GET produktuak/_search
{
  "query": {
    "match": {
      "izena": "koaderno"
    }
  }
}
```

- Aztertu emaitzetako **_score** balioa

2. Bilatu 10 € edo gutxiago balio duten produktuak:

```
GET produktuak/_search
{
  "query": {
    "range": {
      "prezioa": {
        "lte": 10
      }
    }
  }
}
```

3. Konbinatu bi baldintzak bool erabiliz:

```
GET produktuak/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "izena": "koaderno"
          }
        }
      ],
      "filter": [
        {
          "range": {
            "prezioa": {
              "gte": 1,
              "lte": 10
            }
          }
        }
      ]
    }
  }
}
```

— 4. PRAKTIKA

QUERY DSL: LEHEN BILAKETAK

HELBURUA: Query DSL erabiliz **produktuak** indizean **testu-bilaketak**, **prezio-iragazkiak** eta **baldintza konbinatuak** egitea

PAUSOAK:

4. Erantzun:

- a. Zein produktu aurkitzen ditu **match** kontsultak?
- b. Zein produktu aurkitzen ditu **range** kontsultak?
- c. Zein produktu geratzen dira azken **bool** kontsultan?
- d. Zer funtzio betetzen du **must**-ek? Eta **filter**-ek?

5. **Erronka:** Sortu kontsulta bat 10€ eta 50€ arteko produktuak bilatzeko.

ARIKETA GEHIGARRIAK

1. Bilatu **20€ baino garestiagoak** diren produktuak
2. Bilatu **izena** eremuan **mekanikoa** hitza duten produktuak
3. Bilatu **5€ eta 25€ arteko** produktuak.
4. **bool** erabiliz, bilatu izenean **koaderno** duten eta **5€ baino garestiagoak** diren produktuak.
5. **Erronka**: bilatu **Informatika** kategoriako produktuak **term** erabiliz. Zer gertatzen da **kategoria** erabiltzen baduzu? Eta **kategoria.keyword** erabiltzen baduzu?

Elasticsearch: Agregazio Motak



Bucket Agregazioak

Dokumentuak taldetan sailkatu

terms: kategoriak taldekatu **date_histogram:** eguneko taldeak
range: prezio-tarteak
→ SQL GROUP BY baliokidea



Metrika Agregazioak

Talde barruko kalkulu numerikoak

avg: batezbestekoa **max:** maximoa **min:** minimoa
sum: batura **stats:** laburpen osoa **percentiles:** p50/p95/p99
→ SQL SUM(), AVG(), COUNT() baliokidea



Kabiatutako Agregazioak

Bucket barruan metrika kalkulatu

terms (kategoriak) → **avg** (batezbesteko prezioa)
→ **max** (prezio maximoa)
→ **value_count** (produktu kopurua)
→ "Kategoria bakoitzeko batezbesteko prezioa"

bulegoa:	45 prod	avg €3.20	■■■■■■■
tresnak:	23 prod	avg €8.50	■■■■■■■■■■■■■■■■■■■■
papeleria:	12 prod	avg €1.80	■■

— ANALITIKA

Agregazioak: datuak laburbildu eta analizatu

- **Bucket** → dokumentuak taldekatu
terms, date_histogram, range
- **Metrika** → kalkuluak egin
avg, sum, min, max, percentiles
- **Kabiatuak** → talde bakoitzaren barruan kalkuluak egin

04 Kibana

Kibana: Elasticsearch-en interfazea

Kibana Elasticsearch-eko datuak esploratu, bistaratzeak sortu eta administratzeko web-interfazea da.

Tresna nagusiak:

- **Discover** — dokumentuak esploratu eta iragazi
- **Lens** — grafikoak sortu
- **Dashboard** — bistaratzeak panel batean konbinatu
- **Dev Tools** — Query DSL bidez Elasticsearch kontsultatu
- **Stack Management** — indizeak eta ILM kudeatu

Portu lehenetsia: <http://localhost:5601>

IDEIA GAKOA: Kibana interfazea da; Elasticsearch da motorra.
Kibana-k Elasticsearch-era kontsultak bidaltzen ditu atzean.

Kibana: Tresna Nagusiak

Discover

Datu-esploratzea

Bilaketa adierazpenak (KQL)

Denboratarte filtratzea

Eremu aukeraketa

Dokumentu zuzenak ikusi

Visualize / Lens

Bistaratzek sortu

Bar/Line/Pie grafikak

Metrika eta taulak

Drag & drop interfazea

TSVB denb.-lerroak

Dashboard

Ikuspegi globala

Bistaratzek konbinatu

Interaktiboki iragaztea

Auto-freskatzea

Exportatu (PDF, PNG)

Maps

Mapa bistaratzea

Kokapena-oinarritutako analisia

geo_point eta geo_shape

Datu-geruza anitzak

Choropleth mapak

Dev Tools Console

GET /produktuak/_search

```
{ "query": { "match": {  
  "izena": "elasticsearch"  
} } }
```

Elasticsearch APIra zuzeneko sarbidea

Stack Management

Index Pattern / Data View sortu

ILM (Index Lifecycle Mgmt)

Erabiltzaileak eta Rolak

Snapshot Politikak

Alerting eta ML

Threshold-oinarritutako alertak

Anomalia detekzioa (ML)

Email/Slack jakinarazpenak

Watcher (Enterprise)

Kibana: :5601 | Elasticsearch APIrako kotsola integratua

Discover eta KQL

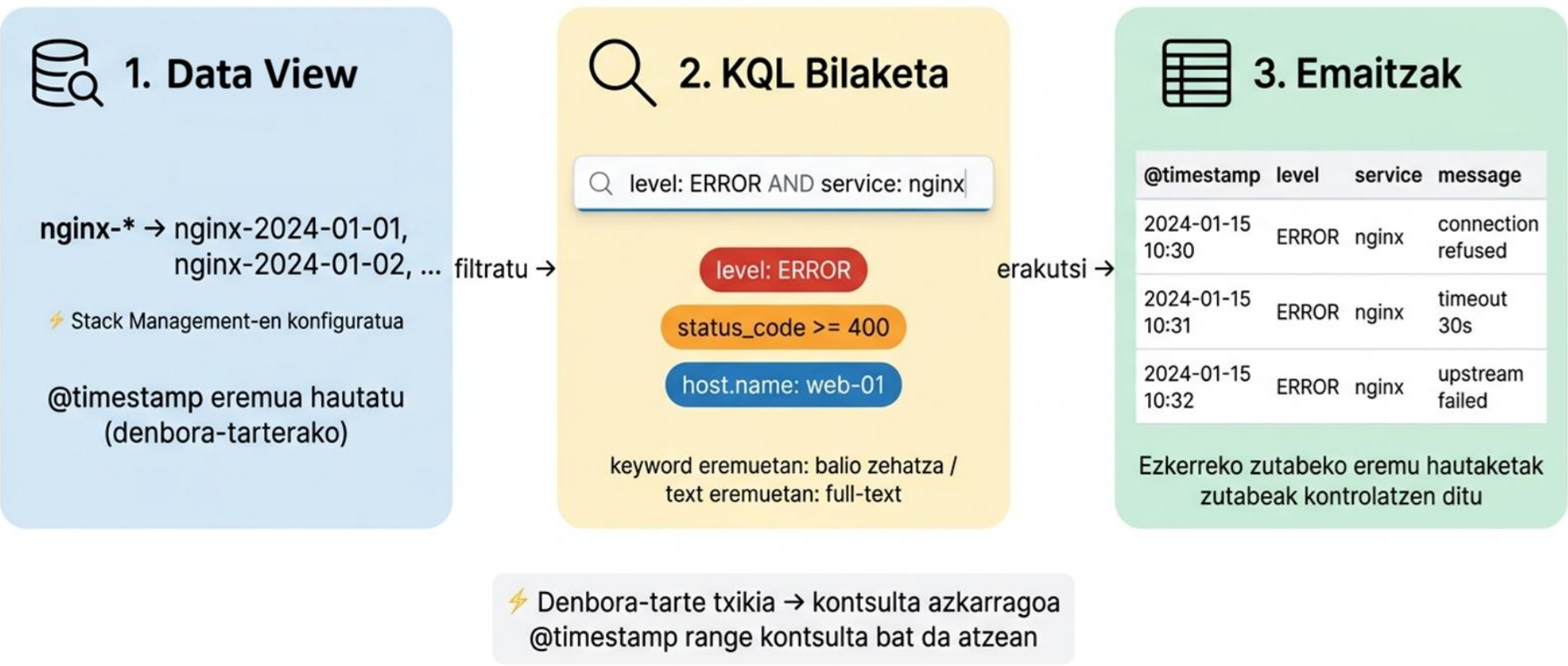
■ Discover

Elasticsearch-eko dokumentuak esploratu eta iragazteko Kibana-ko tresna.

■ Hiru urrats:

Data View → KQL → Emaitzak

Kibana Discover: Nola Funtzionatzen Du



KQL · Kibana Query Language

```
●●● kql.txt python

1  # Balio zehatza
2  kategoria: "Informatika"
3
4  # Zenbakizko tartea
5  prezioa >= 100
6
7  # Baldintzak konbinatu
8  kategoria: "Informatika" AND prezioa < 500
9
10 # Baldintza baztertu
11 NOT kategoria: "Papergintza"
```

AND / OR / NOT

AND → bi baldintzak bete

OR → gutxienez bat bete

NOT → baldintza baztertu

Query DSL → kontsulta osoak eta konplexuagoak

KQL → Kibana-n iragazketa azkarra eta eroso

Discover eta KQL – Datuak esploratzen

HELBURUA: **produktuak** indizeko datuak **Discover-en** esploratu eta **KQL** bidez iragaztea.

PAUSOAK:

1. Ireki Discover.

- Joan **Kibana** → **Discover** atalera.
- Aukeratu **produktuak** indizeari dagokion **Data View-a**.
- Begiratu eskuragarri dauden dokumentuak eta eremuak.

PISTA: **produktuak** Data View-a ez badago, sortu:

- **Stack Management** → **Data Views** → **Create data view**
- Index pattern: **produktuak**
- Produktuek ez dute **@timestamp** eremurik.

2. Prestatu emaitzen taula.

- Gehitu zutabe hauek: **izena**, **kategoria**, **prezioa**

PISTA: Discover-en ezkerreko eremu-zerrendan bilatu eremua eta gehitu taulara.

3. Egin KQL bilaketa hauek:

- Informatika kategoriako produktuak.
- 100€ baino garestiagoak.
- Informatika kategoriakoak **ETA** 500€ baino merkeagoak
- Papergintza kategoriakoak **EZ** diren produktuak.

KQL GOGORATU:

- Balio bat bilatzeko → **eremua**: "**balioa**"
- Konparazioetarako → **>**, **<**, **>=**, **<=**
- Baldintzak konbinatzeko → **AND**, **OR**, **NOT**

4. Zer antzekotasun dago KQL bilaketa hauen eta Query DSL bidez egindako kontsulten artean?

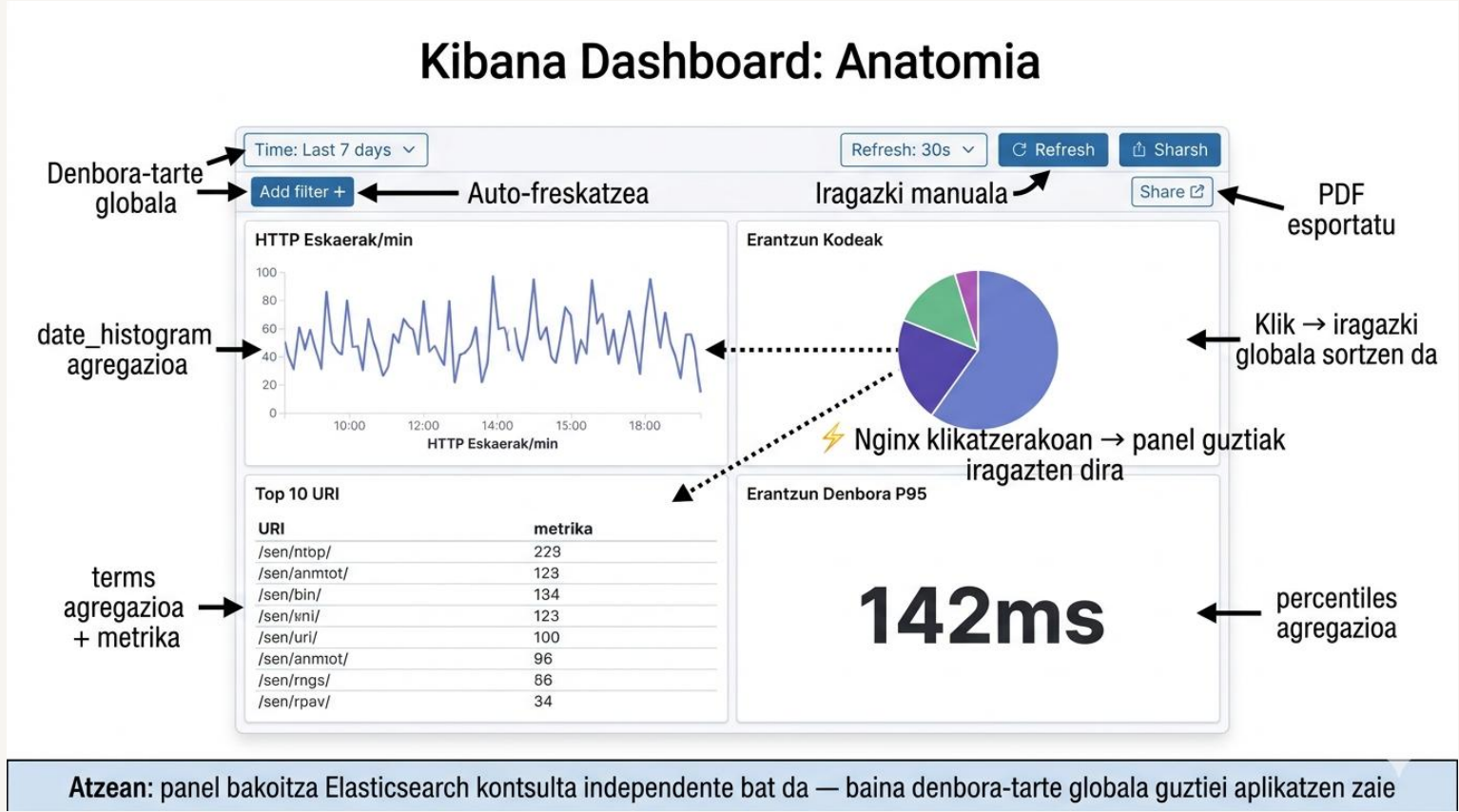
Lens: datuetatik grafikoetara

Kibana Visualization → Elasticsearch-eko datuetatik **bistaratzeak** sortzeko tresna grafikoa.

- **Drag & drop** → eremuak arrastatu, koderik gabe
- **Metrikak** → Count, Average, Sum, Min, Max...
- **Taldekatzea** → kategoriaren, zerbitzuaren... arabera
- **Grafiko motak** → barrak, lerroak, donut, metrikak...

IDEIA GAKOA: Lens-en grafiko baten atzean **Elasticsearch** agregazioak daude.

Dashboard: bistaratzeak panel batean



Dashboard batek bistaratzeak konbinatzen ditu:

- **Panelak konbinatu** → ikuspegi orokorra lortzeko.
- **Denbora-tarte globala** → panel guztiei batera aplikatzen zaie.
- **Iragazketa interaktiboa** → panel batean klik → panel guztiak iragazi.
- **Auto-freskatzea** → datuak automatikoki eguneratu monitorizaziorako.

ATZEAN: panel bakoitzak Elasticsearch kontsultatzen du; iragazki globalak kontsulta guztiei aplikatzen zaizkie.

Lens + Dashboard – Produktuen dashboard-a

HELBURUA: Lens (Visualization) erabiliz bistaratzeak sortu eta **Dashboard** batean konbinatzea.

PAUSOAK:

1. Sortu 1. bistaratzea → Produktuak kategoriaren arabera

- Joan **Visualize Library** → **Create visualization** → **Visualization**
- Erabili **kategoria** eremua
- Erakutsi **produktu kopurua** kategoria bakoitzean
- Erabili **barra-diagrama**
- Gorde: **Produktuak kategoriaren arabera**

PISTA: produktu kopurua kalkulatzeko erabili **Count of records**.

kategoria eremuak produktuak kategoriaka taldekatzeko balio du.

2. Sortu 2. bistaratzea → Batez besteko prezioa kategoriaren arabera

- Erabili **kategoria** → taldekatzea
- Erabili **prezioa** → **Average**
- Erabili **barra-diagrama**
- Gorde: **Batez besteko prezioa kategoriaren arabera**

PISTA: oraingoan ez dugu produktuak zenbatu nahi:

kategoria bakoitzeko **prezioa** eremuaren **batezbestekoa** kalkulatu nahi dugu.

3. Sortu Dashboard-a

- Sortu Dashboard berria (**Dashboards** → **Create a dashboard**)
- Gehitu gordetako **bi bistaratzeak**
- Aldatu panelen **tamaina eta kokapena**
- Gorde: **Produktuen Dashboard-a**

4. Probatu interaktibitatea

- Egin klik grafiko bateko **kategoria batean**
- Begiratu zer gertatzen den beste panelarekin
- Identifikatu sortutako **iragazkia**
- Kendu iragazkia eta egiaztatu emaitza

Lens + Dashboard – Salmenten dashboard-a

HELBURUA: Emandako salmenta-dataset bat (*salmentak_kibana.csv*) Elasticsearch-era inportatu, datuak aztertu, hainbat bistaratze sortu eta **Dashboard interaktibo batean konbinatzea**.

PAUSOAK:

1. Inportatu dataset-a.

Inportatu *salmentak_kibana.csv* Elasticsearch-era (Machine Learning → Data Visualizer → Upload data files).

- Indizearen izena: **salmentak**
- Egiaztatu dokumentuak behar bezala kargatu direla.
- Sortu/egiaztatu **salmentak** indizeari dagokion **Data View**.
- Ziurtatu **data** eremua **Date** motakoa dela.

PISTA: fitxategia kargatzerakoan begiratu Kibana-k eremu bakoitzari esleitutako datu mota.

2. Sortu bistaratzeak

Sortu eta gorde ondorengo bistaratzeak. **Zuek erabaki behar duzue zer eremu eta agregazio erabili behar diren.**

a. Salmentak kategoriaren arabera

Erakutsi kategoria bakoitzean **saldutako unitate guztien kopurua**. Erabili barra-diagrama.

b. Diru-sarrerak hirika

Erakutsi hiri bakoitzak sortutako **diru-sarrera guztiak**. Erabili barra-diagrama.

c. Salmenta-kanalen banaketa

Erakutsi salmenten banaketa **Denda / Web** kanalen artean. Erabili donut edo pie-diagrama

d. Diru-sarreraren bilakaera denboran

Erakutsi **diru-sarrera guztien bilakaera denboran**. Erabili lerro-diagrama eta **data** eremua.

e. Produktuen batez besteko prezioa

Erakutsi produktu bakoitzaren **batez besteko prezioa**. Erabili barra-diagrama.

PISTA: pentsatu kasu bakoitzean zer behar duzuen: **Count, Sum edo Average?**

Eta zein eremuk egiten duen **taldekatzea**.

Lens + Dashboard – Salmenten dashboard-a

HELBURUA: Emandako salmenta-dataset bat (*salmentak_kibana.csv*) Elasticsearch-era inportatu, datuak aztertu, hainbat bistaratze sortu eta **Dashboard interaktibo batean konbinatzea**.

PAUSOAK:

3. Sortu Dashboard-a

Sortu Dashboard berria: **Salmenten Dashboard-a**

Gehitu aurreko **5 bistaratzeak** eta antolatu informazioa modu argian.

Dashboard-ak gutxienez izan behar ditu:

- **5 bistaratzeak**
- **Izenburu argiak**
- **Panelen tamaina eta kokapen egokia**
- Informazioa erraz interpretatzeko antolaketa

4. Probatu Dashboard-aren interaktibitatea

Dashboard-a prest dagoenean: Egin klik hiri batean; begiratu gainerako grafikoak nola aldatzen diren; kendu iragazkia; iragazi **kategoria** edo **kanal** baten arabera eta aztertu emaitza.

5. Azken analisia

1. Zein kategoriatan saldu dira unitate gehien?
2. Zein hirik sortu du diru-sarrera handiena?
3. Zein kanaletan egin dira salmenta gehiago: Web edo Denda?
4. Zein produktuk du batez besteko preziorik handiena?
5. Aukeratu hiri bat eta aplikatu iragazkia. Zer ondorio interesgarri atera dezakezu hiri horretako salmentei buruz?

Dev Tools: Elasticsearch-ekin zuzenean

```
●●● devtools.http python

1  # Bilaketa kontsulta
2  GET /produktuak/_search
3  {
4  "query": { "match": { "izena": "koaderno" } }
5  }
6
7  # Kluster egoera
8  GET /_cluster/health
9
10 # Indizeak ikusi
11 GET /_cat/nodes?v
12
13 # Indize bat sortu / ezabatu
13 PUT /test-indizea
14 DELETE /test-indizea
```

EXEKUTATU

CTRL+ENTER → Kontsulta exekutatzeko lasterbidea.

Emitza → eskuineko panelean JSON formatuan agertzen da

LAGUNTZA

Autocompletatzea → API, indize eta eremuekin laguntza.

Sintaxi-akatsak identifikatzen laguntzen du.

_CAT API

Klusterraren informazioa **formatu irakurgarrian** ikusteko.

/_cat/indices?v → indizeak, dokumentu kopurua, tamaina, egoera...

Discover/Lens → interfaze grafikoa

Dev Tools → Elasticsearch API-a zuzenean

ILM — datuen bizi-zikloa

Indizeen **bizi-zikloa** automatikoki
kudeatzen du.

Arazoa: logak etengabe metatzen dira
→ biltegitratze-beharra eta kostua
handitu.

Soluzioa: datuak erabileraren arabera
fasez aldatu:

Hot → Warm → Cold → Delete

ILM — Index Lifecycle Management



Datu berriak → azkarragoak eta garestiagoak

Datu zaharrak → merkeagoak eta gutxiago erabiltzen dira

ILM Faseak · Ezaugarri eta Helburuak

FASEA	DENBORA *	EZAUGARRIAK	HELBURUA
Hot	0–7 egun	Idazketa + irakurketa aktiboa, errendimendu handia	Azken logak, arazketa
Warm	7–30 egun	Irakurketa nagusiki, erabilera txikiagoa	Datu historikoak
Cold	30–90 egun	Gutxitan kontsultatu, baliabide gutxiago	Artxibatzea, auditoretza
Delete	90–180 egun	Automatikoki ezabatu	Lekua eta kostua murriztu

* Denborak adibideko ILM politikarenak dira; konfiguragarriak dira.

— 8. PRAKTIKA

ILM politika diseinatu

HELBURUA: Diseinatu ILM politika.

Enpresa batek egunero web zerbitzarien logak jasotzen ditu Elasticsearch-en.

- Azken **7 egunetako** logak etengabe kontsultatzen dira.
- **7-30 egunekoak** batzuetan kontsultatzen dira.
- **30-90 egunekoak** auditoretzarako bakarrik gordetzen dira.
- **90 egunetik gorakoak** ez dira behar.

1. Diseinatu ILM politika:

- HOT:
- WARM:
- COLD:
- DELETE:

2. Zergatik ez ditugu log guztiak Hot fasean mantentzen?

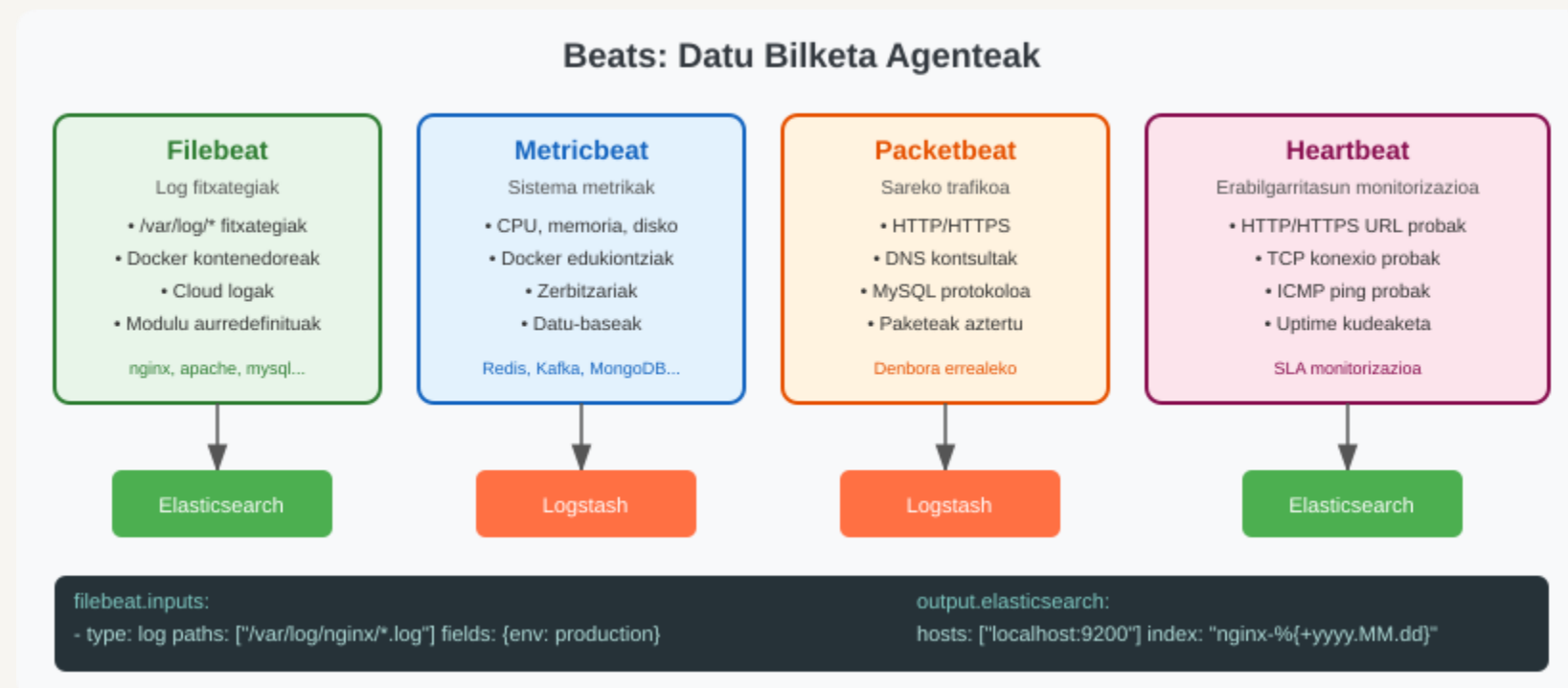
3. Zer arazo izango genuke Delete faserik ez balego?

05

Ingesta: Beats eta Logstash

— AGENTE ARINAK

Beats



- Iturrian instalatutako agente arinak (~50 MB RAM)
- Filebeat (logak), Metricbeat (CPU/RAM), Packetbeat (sarea)
- Heartbeat (uptime), Winlogbeat, Auditbeat
- Datuak Logstash-era edo zuzenean ES-era bidaltzen dituzte

Logstash: testu gordina → eremu egituratuak

Arazoa: `192.168.1.105 ... "GET /api" 200 1234` — testu gordina. Ezin duzu `status\_code: 500` bilatu.

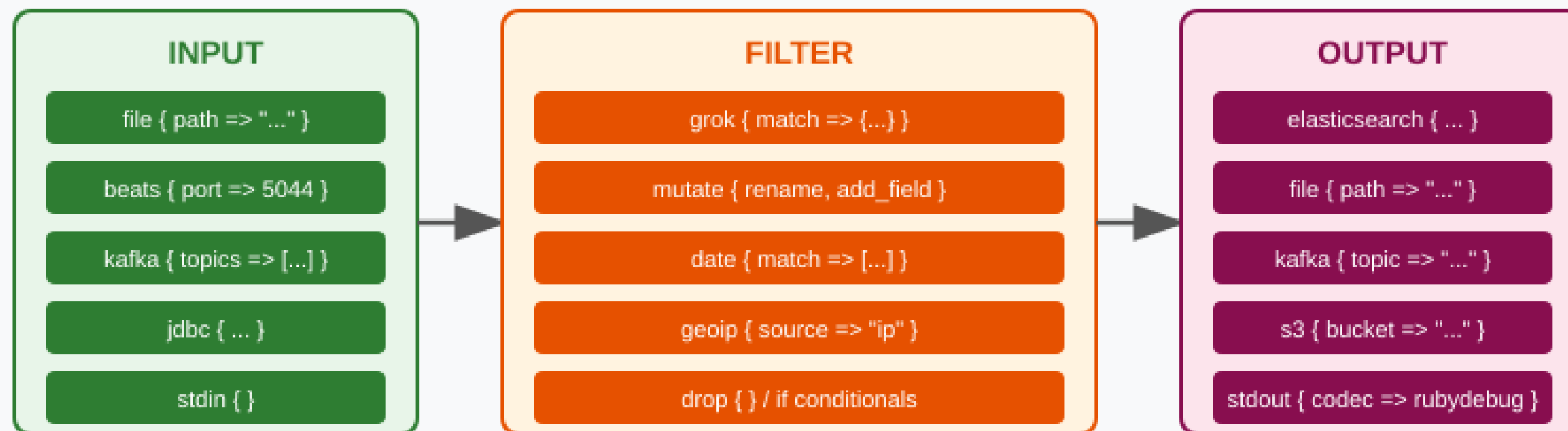
Datua existitzen da, baina ITXITA dago.

Logstash: testu gordina hartu, eremutan banatu, JSON egituratu bihurtu.

Hiru fase: **input** → **filter** → **output** (jaso → eraldatu → bidali).

— PIPELINE

Input → Filter → Output



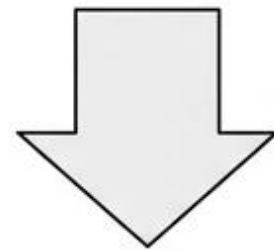
Logstash pipeline: datuak jaso → eraldatu → helmugara bidali

Input (Beats, Kafka, fitxategiak) ·
Filter (grok, date, geoip, mutate) ·
Output (Elasticsearch, S3, Kafka).

grok: Testu Gordina → Eremu Egituratura

```
192.168.1.105 - - [15/Jan/2024:10:30:45 +0100] "GET /api/v1 HTTP/1.1" 200 1234
```

```
%{IP:bezero_ip} %{WORD:metodoa} %{URIPATH:request} HTTP/%{NUMBER:httpversion}  
%{NUMBER:status_code:int} %{NUMBER:bytes:int}
```



grok filter

```
{  
  "bezero_ip": "192.168.1.105",  
  "metodoa": "GET",  
  "request": "/api/v1",  
  "httpversion": "1.1",  
  "status_code": 200,  
  "bytes": 1234  
}
```

%{IP} → IPv4/IPv6 helbidea

%{WORD} → Hitz bat

%{URIPATH} → URL path

%{NUMBER} → Zenbakia

%{GREEDYDATA} → Edozer (⚠ astiro)

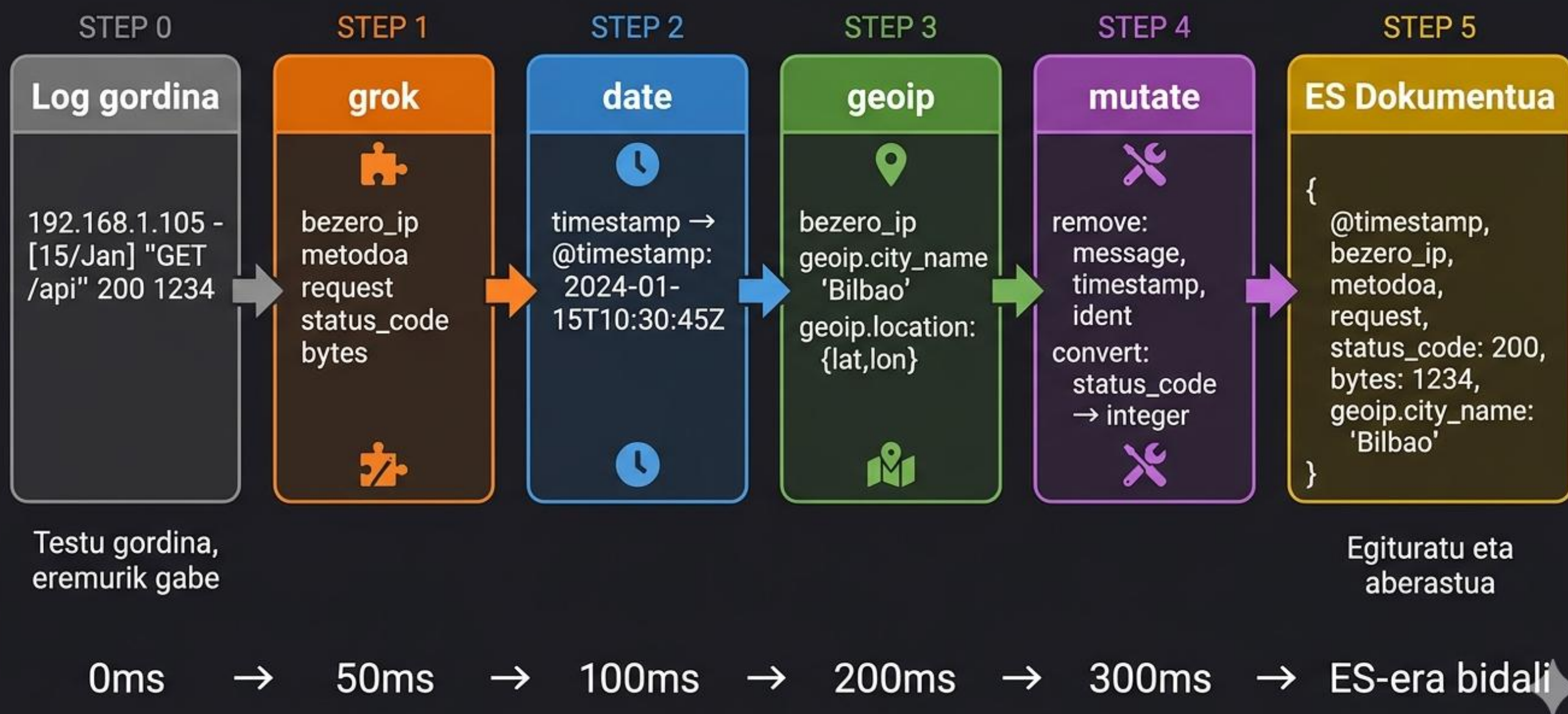
💡 **Kibana Grok Debugger:**
Dev Tools → Grok Debugger.
Pattern garatu aurretik probatu!

— FILTER GAKOA

grok — testu gordina eremu bihurtu

Patroi izendunak: ``%{IP:bezero_ip}`` →
IP helbidea hartu eta eremu batean
gorde. Nginx/Apache:
``%{COMBINEDAPACHELOG}`` → 10+
eremu berehala.

Nginx Log → Filter Katea → ES Dokumentua



— FILTER KATEA

grok → date →
geoip → mutate

Log-lerro baten bizi-zikloa: parseatu
(grok) → timestamp (date) →
kokapena (geoip) → garbitu (mutate)
→ ES dokumentu aberastua.

Filter pitfall klasikoa

date filter ahaztea

Log-ak bere ordua darama, baina ES-ek @timestamp espero du. date filter GABE, @timestamp = Logstash-ek prozesatu duen unea, ez log-aren benetako data → grafikoetan desplazamendua.

IMPORTANTE

Beti jarri date filter-a. Bestela monitorizazio-grafikoetan logak okerreko unean agertzen dira (prozesatu zirenean, ez gertatu zirenean).

Logstash vs Ingest Node

- **Logstash** — grok konplexua, iturri anitz, logika baldintzatua
- **Ingest Node** — ES barnean, eraldaketa sinpleak, nodo gabe
- Erregela: log gordina parseatu → Logstash
- JSON jada egituratua + eraldaketa txikia → Ingest Node

Logstash vs Ingest Node: Noiz Aukeratu?

 Logstash "Pipeline indartsu eta malgua"	 Ingest Node "Sinplea, ES barnean integratua"
✓ Grok parsing konplexua (nginx, apache, syslog) ✓ Iturri anizkoitz (Beats + JDBC + Kafka) ✓ GeoIP, eraldaketa aurreratua ✓ If/else logika baldintzatua ✓ 200+ plugin ekosistema ✓ Persistent Queue (datu galera 0)	✓ JSON logak (jada egituratuta) ✓ Eraldaketa sinpleak (eremu gehitu/kendu) ✓ ES baliabide berdinak erabili ✓ Konfigurazio gutxiago ✓ Doakoa (ES-en barnean)
 1-4GB RAM gehigarria	 Plugin gutxiago, iturri bakarra
Arau praktikoa: Log gordina parseatu behar? → Logstash . JSON jada egituratua? → Ingest Node . Iturri anizkoitz? → Logstash . Eraldaketa sinplea? → Ingest Node	

Gogoan hartzekoak

-
- i* **Alderantzizko indizea** — zergatik azkarragoa SQL LIKE baino

 - ii* **text vs keyword** — term text eremuan ez du ezer aurkitzen

 - iii* **bool**: must (score) vs **filter** (cacheatua, iragazketarako)

 - iv* **Agregazioak**: terms (GROUP BY), date_histogram, percentiles (p99)

 - v* **Logstash**: grok (eremuak) + date (@timestamp ezinbestekoa)

ESKERRAK

Galderak eta eztabaida.

GAIA

2 · Elastic Stack